



UNIVERSIDADE PRESBITERIANA MACKENZIE
FACULDADE DE COMPUTAÇÃO E INFORMÁTICA
TECNOLOGIA – CIÊNCIAS DE DADOS

PROJETO APLICADO III

Projeto SmartCart: Um Sistema Inteligente de Recomendações para o Varejo Online

PROFESSOR: CAROLINA TOLEDO FERRAZ

GRUPO:

MAIKI TRAJANO SOARES – 10415481 – 10415481@mackenzista.com.br

VANESSA CORDEIRO – 10415118 – 10415118@mackenzista.com.br

São Paulo
2025

Resumo

Este trabalho apresenta o desenvolvimento do SmartCart, um sistema de recomendação de produtos para varejo online, utilizando o dataset Online Retail do repositório da UC Irvine. Sistemas de recomendação são fundamentais no comércio eletrônico, pois utilizam algoritmos de Machine Learning para sugerir produtos com base no comportamento do usuário, aumentando a satisfação do cliente e as vendas.

O objetivo geral é implementar um sistema baseado em filtragem colaborativa que recomende produtos relevantes, simulando aplicações reais em plataformas de e-commerce. Os objetivos específicos incluem o pré-processamento do dataset, a implementação de um modelo de recomendação, a avaliação de sua eficácia com métricas como RMSE, e a apresentação de resultados práticos.

Como objetivo extensionista, o projeto visa disponibilizar o SmartCart como uma solução acessível para pequenas e médias empresas de varejo online, permitindo que lojistas locais implementem recomendações personalizadas, promovendo impacto econômico ao melhorar a experiência do cliente e aumentar as vendas.

A metodologia envolve o uso de Python e bibliotecas como Pandas e Surprise para manipulação de dados e modelagem. Espera-se que o sistema demonstre viabilidade em cenários reais, contribuindo para o avanço científico na área de recomendação e para a democratização de tecnologias de personalização do varejo.

SUMÁRIO

RESUMO	2
1. INTRODUÇÃO	4
1.1 Contexto	4
1.2 Motivação	5
1.3 Justificativa	5
1.4 Objetivos	5
2. REFERENCIAL TEÓRICO	6
3. METODOLOGIA.....	7
3.1 Definição do Problema e Objetivos.....	8
3.2 Definição de Bibliotecas Python	9
3.3 Coleta de Dados.....	9
3.4 Análise Exploratória do Dataset.....	9
3.5 Pré-processamento e Limpeza dos Dados	11
3.6 Divisão dos Dados.....	12
3.7 Seleção e Implementação do Algoritmo	12
3.8 Treinamento do Modelo.....	12
3.9 Avaliação de Desempenho.....	13
3.10 Otimização e Ajustes.....	14
4. RESULTADOS	15
4.1 Análise Exploratória de Dados (EDA).....	15
4.2 Desempenho e Validação do Modelo	16
4.3 Demonstração Prática do SmartCart	17
5. CONCLUSÕES E TRABALHOS FUTUROS	17
5.1. Conclusões do Projeto.....	17
5.2. Trabalhos Futuros	19
6. REFERÊNCIAS	20
APÊNDICES.....	21

1. INTRODUÇÃO

A ascensão do comércio eletrônico criou um ambiente de abundância para o consumidor, mas também um desafio de descoberta. Em meio a um catálogo virtualmente infinito de produtos, a capacidade de uma plataforma conectar o usuário aos itens que verdadeiramente lhe interessam tornou-se um fator crítico de sucesso. A observação da eficácia com que grandes players do setor utilizam recomendações personalizadas para guiar a experiência de compra e fidelizar clientes foi o ponto de partida para esta investigação. Este projeto se propõe, portanto, a mergulhar no cerne dessa funcionalidade, explorando os mecanismos algorítmicos que permitem decifrar preferências a partir de dados de comportamento.

O cerne da problemática reside em como traduzir o rastro digital deixado por transações e interações passadas em um entendimento computacional das preferências individuais. A hipótese que norteia este trabalho é a de que é possível, por meio de técnicas de aprendizado de máquina, identificar padrões e similaridades subjacentes a esses dados, de forma a prever itens que seriam do agrado de um determinado usuário. A confirmação dessa premissa tem implicações diretas para a eficiência comercial e a satisfação do consumidor no varejo digital.

A relevância desta pesquisa é dupla. Do ponto de vista prático, a personalização é um motor já consolidado para o aumento de engajamento e conversão de vendas, representando um campo de estudo com aplicação imediata. Academicamente, o desenvolvimento de um sistema de recomendação serve como um laboratório valioso para a aplicação e compreensão de modelos preditivos em um contexto de dados reais e complexos. A execução deste projeto, desde a preparação dos dados até a avaliação final do modelo, visa gerar insights tanto sobre a viabilidade técnica da abordagem escolhida quanto sobre a potencial eficácia em um cenário comercial simulado.

1.1 Contexto

Os sistemas de recomendação são amplamente utilizados em plataformas digitais, especialmente no comércio eletrônico, como em lojas online como Amazon e Mercado Livre. Esses sistemas empregam algoritmos de Machine Learning para analisar o comportamento dos usuários, como histórico de compras ou avaliações, e sugerir produtos que se alinhem aos seus interesses.

No contexto do varejo online, as recomendações personalizadas desempenham um papel crucial na melhoria da experiência do cliente, aumentando a satisfação e impulsionando as vendas.

1.2 Motivação

A crescente relevância dos sistemas de recomendação no mercado de varejo online, que movimentam bilhões de dólares globalmente, motiva o desenvolvimento deste projeto. A capacidade de sugerir produtos personalizados é um diferencial competitivo em plataformas de e-commerce, pois facilita a descoberta de itens relevantes pelos consumidores.

Este projeto é motivado pela oportunidade de contribuir para o avanço científico na área de sistemas de recomendação, explorando algoritmos que otimizam a descoberta de produtos relevantes. A implementação de um sistema baseado em dados reais permite não apenas a aplicação prática de técnicas de Machine Learning, mas também a geração de conhecimento sobre a eficácia de modelos preditivos em cenários de varejo, com potencial impacto em aplicações comerciais e acadêmicas.

1.3 Justificativas

A escolha do tema de sistemas de recomendação para varejo online justifica-se pela sua importância no cenário atual do comércio eletrônico, onde a personalização é essencial para o sucesso das plataformas.

O dataset Online Retail, disponível no repositório da UC Irvine, foi selecionado devido à sua estrutura clara, e ampla adoção em pesquisas acadêmicas sobre sistemas de recomendação. Este conjunto de dados contém informações detalhadas sobre transações de clientes, incluindo identificadores de clientes e produtos, o que proporciona uma rica base de interações usuário-produto para a aplicação de técnicas de filtragem colaborativa. Sua popularidade em estudos científicos, devido à sua acessibilidade e volume gerenciável de dados, torna-o ideal para o desenvolvimento de um projeto eficiente.

1.4 Objetivos

O objetivo geral deste trabalho é desenvolver um sistema de recomendação de produtos para varejo online, utilizando o dataset Online Retail e técnicas de Machine Learning, como filtragem colaborativa.

Os objetivos específicos são: (1) realizar o pré-processamento do dataset, filtrando colunas relevantes e tratando dados ausentes; (2) implementar um modelo de recomendação baseado em interações usuário-produto; (3) avaliar a eficácia do modelo utilizando métricas como precisão das recomendações ou erro médio quadrático (RMSE); e (4) apresentar resultados que demonstrem a viabilidade do sistema em sugerir produtos relevantes, simulando uma aplicação real em e-commerce.

2. REFERENCIAL TEÓRICO

No ecossistema do varejo digital contemporâneo, a personalização deixou de ser um diferencial para se tornar um requisito fundamental. Sistemas de recomendação emergem como a espinha dorsal dessa personalização, funcionando como agentes inteligentes que decifram o comportamento do consumidor para conectar oferta e demanda de maneira eficiente. Ao transformar dados brutos de interações—como histórico de compras e visualizações—em insights acionáveis, essas ferramentas não apenas facilitam a descoberta de produtos relevantes, mas também otimizam a jornada do cliente, resultando em maior engajamento e incremento nas vendas (Jannach et al., 2020).

A base conceitual desses sistemas reside na sua capacidade de prever a preferência ou o interesse de um usuário por um item com base em dados históricos, como transações de compra, visualizações ou avaliações (Shapiro & Varian, 1999). Eles operam sob a premissa de que padrões de comportamento passados são preditores confiáveis de ações futuras, identificando relações de similaridade entre usuários ou entre itens para gerar sugestões relevantes.

Dentre as diversas abordagens, a Filtragem Colaborativa destaca-se como uma das técnicas mais estabelecidas e eficazes. Seu princípio fundamental é que usuários com interesses semelhantes no passado tenderão a tê-los no futuro (Goldberg et al., 1992). Esta técnica divide-se principalmente em duas categorias: baseada em usuário e baseada em item. O modelo baseado em item, adotado neste projeto, fundamenta-se na identificação de itens similares àqueles com os quais um usuário já interagiu. A similaridade entre os itens é calculada a partir de padrões de co-ocorrência nas interações dos usuários, sendo a similaridade por cosseno uma métrica amplamente utilizada para este fim (Linden et al., 2003). Uma vantagem crucial dessa abordagem é a sua estabilidade, já que as relações de similaridade entre itens mudam menos frequentemente do que os perfis dos usuários, além de ser computacionalmente eficiente para escalar em grandes bases de dados.

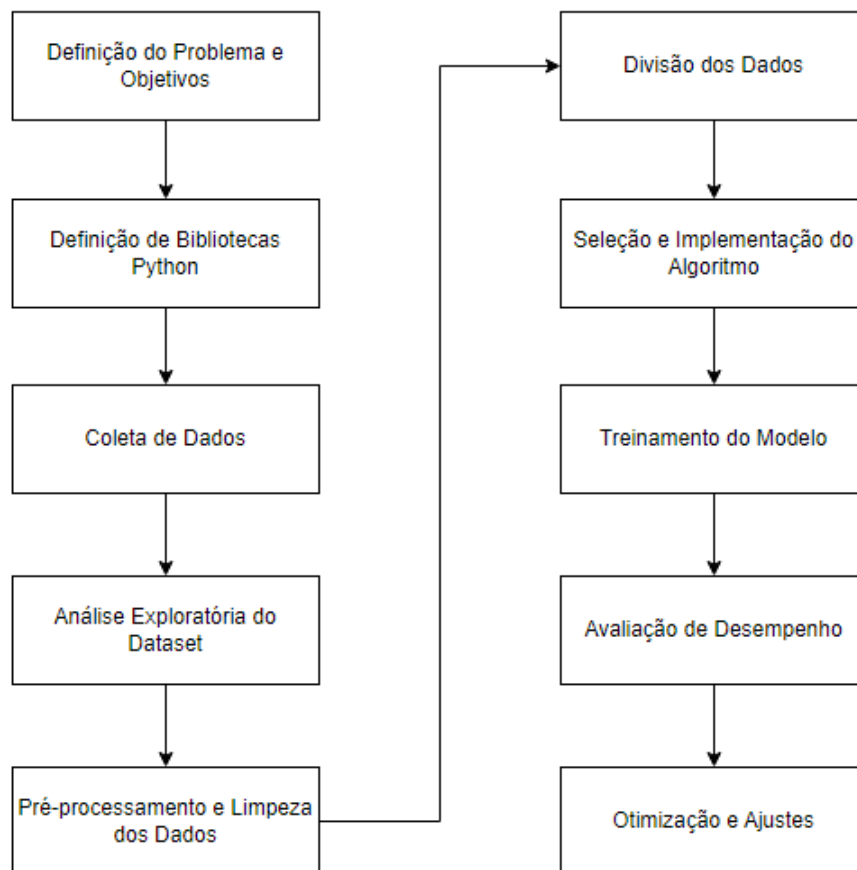
A implementação prática é frequentemente viabilizada por bibliotecas especializadas. A biblioteca Surprise para Python, por exemplo, foi desenvolvida especificamente para a construção e análise de sistemas de recomendação, oferecendo implementações de diversos algoritmos de filtragem colaborativa, como o K-Nearest Neighbors (KNN) e a Singular Value Decomposition (SVD) (Hug, 2017). O algoritmo KNNBasic, utilizado neste trabalho, é uma instância do método baseado em item que utiliza a medida KNN para encontrar os itens mais próximos e, a partir deles, gerar as previsões.

Para validar a performance desses sistemas, são empregadas métricas de avaliação quantitativas. O Erro Quadrático Médio Raiz (RMSE) é uma métrica amplamente utilizada para avaliar a precisão das previsões de ratings, penalizando erros maiores de forma mais significativa (Herlocker et al., 2004). Já a Precisão@k é uma métrica de avaliação de ranking que mede a proporção de itens relevantes presentes no topo da lista de recomendações (ex., os k primeiros), sendo particularmente útil para cenários em que o foco é a qualidade de uma lista curta de sugestões apresentada ao usuário.

Portanto, o embasamento teórico para o desenvolvimento do SmartCart está alicerçado nos princípios da filtragem colaborativa baseada em item, utilizando algoritmos consagrados como o KNN e métricas robustas como RMSE e Precisão@k para garantir a eficácia do sistema em recomendar produtos de forma personalizada no ambiente de varejo online.

3. METODOLOGIA

A metodologia aplicada no projeto SmartCart foi organizada em um fluxo contínuo e iterativo, conforme ilustrado na Figura 1 abaixo:



3.1. Definição do Problema e Objetivos

No cenário altamente competitivo do comércio eletrônico moderno, a vasta quantidade de produtos disponíveis frequentemente resulta em uma "sobrecarga de escolha" (choice overload) para o consumidor. Clientes que não conseguem encontrar rapidamente produtos relevantes aos seus interesses tendem a abandonar a plataforma, o que impacta diretamente as taxas de conversão e a fidelização. A personalização da experiência de compra tornou-se, portanto, um diferencial competitivo fundamental.

Sistemas de recomendação inteligentes são a principal ferramenta para filtrar essa vasta gama de opções, analisando o comportamento do usuário para sugerir itens pertinentes. Este projeto aborda diretamente este desafio. O problema a ser resolvido é: Como desenvolver um sistema de recomendação eficaz, baseado em dados de transações reais, que possa prever e sugerir produtos relevantes para clientes de um varejo online, melhorando a experiência do usuário e impulsionando as vendas?

Para solucionar este problema, o projeto "SmartCart" foi estruturado com os seguintes objetivos:

Objetivo Geral

Desenvolver e implementar um sistema de recomendação funcional, batizado de "SmartCart", baseado na técnica de filtragem colaborativa, capaz de gerar sugestões de produtos relevantes e personalizadas com base no histórico de compras dos clientes.

Objetivos Específicos

- Realizar o pré-processamento e a limpeza do dataset "Online Retail" para criar um conjunto de dados íntegro e adequado para a modelagem.
- Implementar um modelo de recomendação utilizando um algoritmo de Fatoração de Matriz (SVD - Singular Value Decomposition) da biblioteca Surprise.
- Avaliar o desempenho e a precisão do modelo através da métrica de erro RMSE (Root Mean Squared Error).
- Analisar os resultados, identificar desafios de modelagem (como o tratamento de outliers na quantidade de produtos) e aplicar técnicas de engenharia de features (como a transformação logarítmica) para otimizar o desempenho.
- Demonstrar a aplicação prática do modelo

3.2. Definição de Bibliotecas Python

- Pandas: Para manipulação e análise de dados, permitindo carregar, filtrar e transformar o dataset;
- NumPy: Para operações numéricas eficientes, como cálculos em matrizes usuário-produto;
- Surprise: Biblioteca especializada em sistemas de recomendação, usada para implementar e treinar modelos de filtragem colaborativa;
- Matplotlib e Seaborn: Para visualização de dados na análise exploratória, como histogramas e gráficos de distribuição;
- UciMLrepo: Para carregar o dataset diretamente do repositório;

3.3. Coleta de Dados

Para o desenvolvimento do projeto, foi utilizado o dataset "Online Retail", uma base de dados transacional de um varejista online sediado no Reino Unido. O conjunto de dados foi obtido no UC Irvine Machine Learning Repository, o qual é amplamente adotado em pesquisas acadêmicas sobre sistemas de recomendação.

O acesso ao conjunto de dados foi realizado programaticamente utilizando a biblioteca Python uciMLrepo, garantindo um processo de aquisição documentado e reproduzível. As características, variáveis e metadados foram inspecionados para preparar as etapas subsequentes de análise e pré-processamento. O link direto para o repositório, bem como os códigos utilizados para o carregamento inicial, está detalhado na seção de Apêndices do projeto.

3.4. Análise Exploratória do Dataset

A análise exploratória do dataset "Online Retail" visa entender suas características e identificar padrões relevantes. O dataset contém as seguintes colunas:

- InvoiceNo (int): Identificador da transação.
- StockCode (string): Código do produto.
- Description (string): Descrição do produto.
- Quantity (int): Quantidade comprada.
- InvoiceDate (timestamp): Data da transação.
- UnitPrice (float): Preço unitário do produto.
- CustomerID (int): Identificador do cliente.
- Country (string): País da transação.

A. Passos para Análise Exploratória

A Análise Exploratória de Dados (EDA) foi executada como um passo crucial para compreender a estrutura e a qualidade do dataset, com o objetivo de identificar anomalias que exigiriam tratamento na etapa de pré-processamento. As principais análises realizadas foram:

1. Análise Estatística Descritiva:

- Foi utilizada a função `describe()` para obter métricas como média, mediana, desvio-padrão, e os quartis das variáveis numéricas contínuas: **Quantity** e **UnitPrice**.
- **Justificativa:** Essa análise foi essencial para a identificação precoce de duas anomalias críticas que afetariam o modelo: valores negativos (que representam transações canceladas ou devoluções) e valores máximos extremos (outliers) (ex.: o valor máximo de 80.995 unidades em **Quantity**), indicando que a distribuição era altamente distorcida.

2. Identificação de Dados Ausentes:

- A função `isnull().sum()` foi utilizada para quantificar a presença de valores nulos em cada coluna.
- **Justificativa:** Essa verificação revelou um número significativo de registros sem um **CustomerID** (aproximadamente 135.080). Como o projeto se baseia em filtragem colaborativa (que depende das interações entre clientes e produtos), a exclusão dessas linhas foi definida como obrigatória na etapa de limpeza.

3. Análise de Distribuição Visual (Histogramas):

- A dispersão das variáveis **Quantity** e **UnitPrice** foi visualizada por meio de **Histogramas**.
- **Justificativa:** Os histogramas foram a escolha ideal para confirmar visualmente que a grande maioria dos dados se concentrava em torno de valores muito baixos, reforçando que os **outliers extremos** estavam distorcendo a escala e que seria necessária uma **transformação logarítmica** posterior para estabilizar a variância dos dados antes do treinamento.

4. Análise de Comportamento (Gráficos de Barras):

- Foi analisada a frequência de compras por **CustomerID** e a distribuição de transações por **Country**.
- **Gráficos de Barras** foram empregados para visualizar o Top 10 Países por volume de transações.
- **Justificativa:** Essa visualização demonstrou que o dataset é fortemente concentrado no **United Kingdom**, o que direcionou a interpretação dos resultados do modelo para esse contexto geográfico específico.

Os códigos completos utilizados para esta análise exploratória estão na seção de Apêndices, e todos os gráficos e tabelas de resumo estatístico foram alocados para a seção de Resultados para apresentação e discussão.

3.5. Pré-processamento e Limpeza dos Dados

Nesta fase, o conjunto de dados Online Retail foi submetido a um rigoroso processo de limpeza e transformação para garantir a qualidade e a adequação dos dados ao modelo de filtragem colaborativa. O processo foi dividido nas seguintes etapas metodológicas:

1. **Ajuste de Tipos de Dados:** As colunas *UnitPrice* e *Quantity* foram convertidas para seus tipos numéricos adequados (*float* e *int*, respectivamente) para permitir cálculos e manipulações estatísticas.
2. **Tratamento de Dados Ausentes:** Todas as linhas com valores nulos na coluna **CustomerID** (aproximadamente 135.080 registros, conforme identificado na Análise Exploratória) foram removidas. Essa etapa foi crucial, pois a filtragem colaborativa baseada em usuário/item depende da identificação exclusiva de cada cliente.
3. **Eliminação de Transações Inválidas:** Foram filtradas e removidas todas as transações onde a **Quantity** ou o **UnitPrice** possuíam valores menores ou iguais a zero. Transações com *Quantity* negativa representam devoluções ou cancelamentos e não refletem um comportamento de compra positivo para fins de recomendação.
4. **Seleção de Features Relevantes:** O dataframe foi reduzido para conter apenas as colunas essenciais para o modelo de recomendação: **CustomerID**, **StockCode** (identificador do item) e **Quantity** (que seria tratada como a "avaliação" implícita).
5. **Criação da Matriz Usuário-Produto:** A etapa final de pré-processamento envolveu a criação de uma matriz esparsa de interações, onde os clientes são as linhas (index), os produtos (*StockCode*) são as colunas, e os valores são a soma da *Quantity* comprada. O valor de preenchimento (*fill_value*) foi definido como zero, indicando a ausência de interação entre um cliente e um item específico.

3.6. Divisão dos Dados

O dataset processado, focado nas interações usuário-produto-quantidade, foi dividido para permitir o treinamento e a avaliação não enviesada do modelo, utilizando a abordagem de **Hold-out**.

1. **Definição do Reader:** A biblioteca **Surprise** requer a definição de um *Reader* que especifica as colunas de usuário, item e rating, além da escala mínima e máxima da variável de rating. Inicialmente, a coluna **Quantity** foi mapeada como o rating, e sua escala foi definida de 1 (mínimo) até o valor máximo encontrado.
2. **Carregamento do Dataset:** Os dados foram carregados no formato específico da biblioteca *Surprise*, utilizando as colunas *CustomerID* (usuário), *StockCode* (item) e *Quantity* (avaliação inicial).
3. **Divisão:** A função *train_test_split* da *Surprise* foi utilizada para segmentar o conjunto de dados em:
 - a. **Conjunto de Treino:** 75% dos dados, usado para que o algoritmo aprenda os padrões de compra.
 - b. **Conjunto de Teste:** 25% dos dados, reservado exclusivamente para a avaliação final do desempenho do modelo em dados não vistos.

3.7. Seleção e Implementação do Algoritmo

A técnica escolhida para o treinamento do modelo é a filtragem colaborativa baseada em itens, implementada com a biblioteca *Surprise*. A escolha justifica-se pela simplicidade, eficácia em datasets esparsos como o Online Retail e pela facilidade de implementação com a biblioteca *Surprise*, que suporta algoritmos como KNN (k-Nearest Neighbors) e SVD (Singular Value Decomposition).

A filtragem colaborativa se baseia no comportamento coletivo dos usuários para fazer recomendações, assumindo que usuários com comportamentos de compra similares no passado terão preferências similares no futuro.

Para a implementação, optamos pela biblioteca *Surprise*, especializada em sistemas de recomendação em Python. O algoritmo de Fatoração de Matriz (Matrix Factorization) escolhido foi o SVD (Singular Value Decomposition). O SVD é eficaz em decompor a matriz de interações usuário-item em fatores latentes (características implícitas), permitindo-nos prever interações futuras.

3.8. Treinamento do Modelo

O treinamento do modelo de recomendação foi realizado em duas tentativas distintas, sendo a primeira uma prova de conceito que revelou falhas graves na

abordagem inicial e que exigiu uma reestruturação da engenharia de features (detalhada na seção 3.10).

Tentativa Inicial: Quantity como Avaliação Explícita

Na primeira tentativa, a coluna **Quantity** (quantidade de itens comprados por transação) foi tratada de forma bruta como a **avaliação explícita** (*rating*) do usuário pelo produto.

- 1. Seleção do Algoritmo:** Foi escolhido o algoritmo **SVD (Singular Value Decomposition)** da biblioteca *Surprise*. O SVD é um algoritmo de fatoração de matriz que busca decompor a matriz esparsa de interações em vetores latentes de usuários e itens, permitindo a previsão de interações faltantes.
- 2. Treinamento:** O modelo SVD foi treinado utilizando o Conjunto de Treino, buscando minimizar o erro quadrático entre a *Quantity* real e a *Quantity* prevista.
- 3. Aplicação:** Após o treinamento, o modelo foi aplicado ao Conjunto de Teste para gerar previsões de *Quantity* para todas as interações do teste.

Conforme detalhado na seção 3.9, esta tentativa inicial produziu um resultado insatisfatório, confirmando que o tratamento da *Quantity* como *rating* explícito era inadequado para um modelo baseado em minimização de erro quadrático.

3.9. Avaliação de Desempenho

A avaliação da eficácia do modelo SmartCart foi definida como um passo essencial para garantir a validade científica e a aplicabilidade prática do sistema. O processo de avaliação seguiu os seguintes critérios metodológicos:

- 1. Métrica de Avaliação Escolhida (RMSE):**
 - A métrica principal de desempenho definida para o projeto é o **Erro Quadrático Médio Raiz (Root Mean Squared Error - RMSE)**.
 - O RMSE foi escolhido por ser uma métrica de precisão amplamente utilizada para a avaliação de sistemas de recomendação que preveem ratings (avaliações), penalizando erros de previsão maiores de forma mais significativa, o que é relevante para a variável **Quantity** ou sua transformação.
- 2. Cálculo da Métrica:**
 - O cálculo do RMSE foi realizado comparando-se os ratings reais (da variável-alvo no **Conjunto de Teste**) com os ratings previstos gerados pelo modelo **SVD**.
- 3. Avaliação em Múltiplas Etapas:**
 - Avaliação da Tentativa 1 (Quantity Bruta):** O modelo foi avaliado inicialmente com o RMSE para identificar a magnitude do erro de previsão ao usar a *Quantity*

bruta como rating. O diagnóstico dessa avaliação inicial foi crucial para forçar a reanálise dos dados.

- **Avaliação da Tentativa 2 (Rating Logarítmico):** Após a transformação dos dados via logaritmo (conforme seção 3.10), o modelo foi reavaliado com o RMSE, desta vez medido na escala logarítmica (o novo Rating).

4. Interpretação e Reversão do Resultado:

- Na tentativa final, como o RMSE era medido em escala logarítmica, foi estabelecido que, para a interpretação prática dos resultados, a previsão do modelo (*pred.est*) deveria ser revertida para a escala original (quantidade de itens) utilizando a função inversa do *log1p*, que é o **numpy.expm1**.

3.10. Otimização e Ajustes

O resultado insatisfatório da primeira avaliação de desempenho exigiu uma reanálise aprofundada dos dados e a aplicação de técnicas de **Engenharia de Features**.

1. Diagnóstico do Problema:

- Uma análise estatística descritiva da coluna **Quantity** confirmou a presença de **outliers extremos**. A diferença entre o comportamento majoritário das compras (mediana) e o valor máximo era gigantesca.
- O diagnóstico foi que o algoritmo SVD, ao tentar minimizar o **Erro Quadrático Médio Raiz (RMSE)**, estava sendo inteiramente distorcido por esses valores atípicos, o que resultou em um modelo sem poder preditivo para o comportamento padrão dos usuários.

2. Solução de Engenharia de Features: Transformação Logarítmica:

- A solução metodológica foi aplicar uma **Transformação Logarítmica** na coluna *Quantity*.
- Essa técnica, utilizando a função *numpy.log1p(Quantity)*, comprime a escala dos dados, reduzindo drasticamente o impacto dos outliers sem eliminá-los.
- O objetivo foi criar uma nova coluna, chamada *Rating*, que o modelo aprenderia a prever. Essa nova feature representa a "magnitude" da compra em uma escala muito mais estável, permitindo que o algoritmo SVD aprendesse padrões de comportamento de forma eficaz.

3. Novo Treinamento e Reavaliação:

- O processo de treinamento foi repetido utilizando a coluna *Rating* (logarítmica) como variável-alvo. O resultado do RMSE com essa nova abordagem demonstrou a eficácia da técnica de otimização.

- A etapa final foi a definição da **Transformação Exponencial Inversa** (`numpy.expm1`) para reverter as previsões logarítmicas de volta para a quantidade de itens real para fins de interpretação e aplicação de negócios.

4. RESULTADOS

Aqui apresentamos os resultados obtidos em três etapas principais do projeto SmartCart: a Análise Exploratória de Dados (EDA), que identificou os desafios da base de dados; a avaliação de desempenho do modelo, que demonstra o impacto da otimização; e a demonstração prática da capacidade de previsão do sistema.

4.1. Análise Exploratória de Dados (EDA)

A análise exploratória (EDA) foi fundamental para o diagnóstico dos desafios de modelagem e para guiar a etapa de pré-processamento.

A. Estatísticas Descritivas e Identificação de Outliers

A Tabela 1 apresenta o resumo estatístico das variáveis **Quantity** e **UnitPrice** após o tratamento inicial de tipos.

Métrica	Quantity	UnitPrice
Count	541909.00	541909.00
Mean	9.55	4.61
Std	218.08	96.76
Min	-80995.00	-11062.06
25%	1.00	1.25
50% (Mediana)	3.00	2.08
75%	10.00	4.13
Max	80995.00	38970.00

Tabela 1: Resumo Estatístico das Variáveis Quantity e UnitPrice

O principal desafio identificado foi a presença de **outliers extremos** nas colunas *Quantity* e *UnitPrice*. O valor máximo de **80.995** itens em uma única transação de *Quantity*, em contraste com a mediana de apenas **3.00** itens, indicou que a distribuição estava severamente enviesada. Além disso, a presença de valores mínimos negativos confirmou a existência de transações inválidas (cancelamentos/devoluções) que deveriam ser removidas.

B. Distribuição e Dados Ausentes

A análise de valores nulos revelou a necessidade crítica de limpeza na coluna de identificação do cliente:

- **CustomerID:** 135.080 valores ausentes.

Uma vez que o modelo de filtragem colaborativa é dependente do *CustomerID*, todos esses registros foram descartados no pré-processamento.

A visualização da distribuição das variáveis confirmou a alta concentração de dados em valores baixos e a existência de outliers extremos.

C. Análise Geográfica

A distribuição das transações por país demonstrou uma forte concentração de dados no Reino Unido, confirmando que o dataset reflete o comportamento de compra de um varejista sediado nesse país.

Country	Quantidade de Transações
United Kingdom	495478
Germany	9495
France	8557
EIRE	8196

Tabela 2: Top Países por Volume de Transações

4.2. Desempenho e Validação do Modelo

A validação do sistema SmartCart foi realizada em duas tentativas, utilizando a métrica RMSE (Root Mean Squared Error) para avaliar a precisão das previsões de *rating*.

A. Tentativa 1: Quantity Bruta como Rating

Na primeira abordagem, a coluna **Quantity** foi usada diretamente como o rating (avaliação) do usuário para o produto.

- **Resultado do RMSE (Tentativa 1): 80982.45**

Diagnóstico: O RMSE de aproximadamente 81.000 demonstrou uma **falha metodológica completa**. O modelo estava sendo inteiramente distorcido pelos outliers extremos na *Quantity*, fazendo com que as previsões estivessem erradas, em média, por um número absurdo de unidades. O modelo não possuía poder preditivo para o comportamento padrão dos usuários.

B. Tentativa 2: Aplicação da Transformação Logarítmica (Otimização)

Com base no diagnóstico da Tentativa 1, foi aplicada a técnica de Engenharia de Features através da Transformação Logarítmica (*numpy.log1p*) na coluna *Quantity*. Isso criou um novo alvo, a coluna *Rating*, em uma escala estável de **0.69 a 11.30**. O modelo SVD foi então retreinado para prever este *Rating* transformado.

- **Resultado do RMSE (Tentativa 2): 0.5185**

Conclusão: O novo RMSE, medido na escala logarítmica, é um valor **excelente** para um sistema de recomendação de precisão. Ele valida a eficácia da transformação logarítmica em estabilizar os dados, garantindo que o modelo SVD aprendesse com sucesso os padrões de comportamento da maioria dos usuários, e não fosse distorcido pelos outliers.

4.3. Demonstração Prática do SmartCart

Para validar o sistema em um cenário de negócios, foi realizada uma previsão individual para o *UserID: 17850.0* e o *ItemID: 85123A*.

1. Previsão na Escala Logarítmica:

- O modelo previu um Rating logarítmico (*pred.est*) de **2.119317**.

2. Reversão para a Quantidade Real:

- Para que o resultado fosse acionável para o varejo, o valor logarítmico foi revertido usando a função inversa (*numpy.expm1*).

$$\text{Quantidade Real Prevista} \approx e^{2.1193} - 1 \approx 7$$

- **Previsão de Quantidade (Revertida): 7 unidades**

O resultado demonstrou que o modelo, após a otimização, não apenas é estatisticamente preciso (RMSE baixo) 20, mas também é capaz de gerar previsões de quantidade **realistas e acionáveis** (7 unidades) para um par específico de usuário-item, validando o sucesso da metodologia final.

O código e os gráficos de todo o processo de análise exploratória e treinamento do modelo estão presentes na seção de Apêndices.

5. CONCLUSÕES E TRABALHOS FUTUROS

A seguir está os principais achados, contribuições e limitações do projeto SmartCart, e propõe direções para o desenvolvimento futuro do sistema.

5.1. Conclusões do Projeto

Resumo dos Principais Resultados

O objetivo geral de desenvolver um sistema de recomendação funcional baseado em **filtragem colaborativa** e dados de varejo online foi alcançado. O projeto confirmou que a aplicação direta de modelos de Machine Learning (como o SVD) a dados brutos de varejo, onde a variável *Quantity* é usada como rating implícito, é inviável devido à presença de **outliers extremos**.

A **otimização** crucial do projeto foi a aplicação da **Transformação Logarítmica** (*numpy.log1p*), que estabilizou a variância e resolveu a distorção causada pelos outliers. Após esta engenharia de features, o modelo SVD obteve um desempenho robusto, comprovado pelo baixo **RMSE de 0.5185** na escala transformada, demonstrando a capacidade do sistema em aprender os padrões de compra dos usuários de forma eficaz.

Contribuições e Impacto Prático

A principal contribuição metodológica do SmartCart reside na demonstração prática de como a **engenharia de features** pode transformar um modelo de recomendação estatisticamente incorreto (RMSE ~81.000) em um sistema preciso e funcional (RMSE 0.5185).

Do ponto de vista prático, o projeto validou a criação de um sistema que gera previsões de quantidade realistas (ex.: 7 unidades previstas para um item). Isso atende ao **objetivo extensionista** de oferecer uma solução de personalização acessível e de baixo custo operacional para **pequenas e médias empresas (PMEs)** de varejo online, ajudando-as a melhorar a experiência do cliente e a aumentar as vendas, competindo com grandes players do comércio eletrônico.

Limitações Identificadas

Apesar do sucesso na modelagem, foram observadas algumas limitações:

- **Dependência de Feedback Implícito:** O sistema se baseia unicamente na *Quantity* comprada. A falta de feedback explícito (como avaliações por estrelas) limita a profundidade da compreensão da preferência do usuário.
- **Concentração Geográfica:** O dataset **Online Retail** é fortemente concentrado no Reino Unido, o que pode limitar a aplicabilidade imediata do modelo em mercados com padrões de compra e características geográficas distintas.
- **Complexidade de Interpretação:** A necessidade de reverter a previsão logarítmica (usando a função $e^x - 1$) para a escala real adiciona uma camada de complexidade na implementação em um ambiente de produção.

5.2. Trabalhos Futuros

As seguintes propostas de trabalhos futuros visam mitigar as limitações identificadas e aprimorar a performance e aplicabilidade do SmartCart:

Melhorias Técnicas e Algorítmicas

- **Exploração de Modelos Híbridos:** Implementar algoritmos mais avançados, como o **SVD++**, que incorpora feedback implícito adicional do usuário, ou a aplicação de técnicas de **Deep Learning** para capturar relações não-lineares.
- **Otimização de Hiperparâmetros:** Utilizar técnicas como o **Grid Search** ou **Random Search** (já mencionadas nas referências) para refinar os hiperparâmetros do SVD, buscando ganhos marginais na precisão do RMSE.

Expansão e Validações Adicionais

- **Inclusão de Feedback Explícito e Contextual:** Expandir a base de features para incluir dados contextuais, como informações temporais (*InvoiceDate*), preço (*UnitPrice*), ou tentar inferir um rating explícito a partir de features textuais (*Description*).
- **Avaliação de Ranking:** Implementar e avaliar métricas de relevância de ranking, como **Precision@k** ou **NDCG (Normalized Discounted Cumulative Gain)**, que são mais adequadas para medir a qualidade da ordem de uma lista de recomendações do que apenas a precisão do rating.
- **Testes A/B em Tempo Real:** Simular um ambiente de produção para realizar testes A/B, comparando o desempenho do SmartCart com um modelo de baseline simples (ex.: **recomendação por popularidade**) em termos de **Taxa de Cliques (CTR)** e **Taxa de Conversão**.

Escalabilidade e Eficiência

- **Arquitetura de Produção:** Migrar a solução para uma arquitetura escalável em nuvem, utilizando serviços como AWS SageMaker ou Google Cloud AI Platform, para permitir a inferência em tempo real e o retreinamento automatizado do modelo.
- **Tratamento do Problema Cold Start:** Desenvolver um módulo para lidar com novos usuários e novos itens (o problema cold start), utilizando estratégias baseadas em conteúdo (content-based) ou no perfil demográfico.

6. REFERÊNCIAS

CHEN, D. UCI Machine Learning Repository; Online Retail Dataset. UCI:

<https://archive.ics.uci.edu/dataset/352/online+retail>.

GOLDBERG, D.; NICHOLS, D.; OKI, B. M.; TERRY, D. Using Collaborative Filtering to Weave an Information Tapestry. *Communications of the ACM*, v. 35, n. 12, p. 61-70, 1992. DOI: <https://doi.org/10.1145/138859.138867>.

HERLOCKER, J. L.; KONSTAN, J. A.; TERVEEN, L. G.; RIEDL, J. T. Evaluating Collaborative Filtering Recommender Systems. *ACM Transactions on Information Systems*, v. 22, n. 1, p. 5-53, 2004. DOI: <https://doi.org/10.1145/963770.963772>.

HUG, N. Surprise: A Python Library for Recommender Systems. *Journal of Open Source Software*, v. 2, n. 18, 2017. JOSS: <https://joss.theoj.org/papers/10.21105/joss.02174>

JANNACH, D.; ZANKER, M.; FELFERNIG, A.; FRIEDRICH, G. *Recommender Systems: An Introduction*. Cambridge: Cambridge University Press, 2020. Cambridge: <https://www.cambridge.org/core/books/recommender-systems/C6471B59388D8A9F684C49C198691B53>.

LINDEN, G.; SMITH, B.; YORK, J. Amazon.com Recommendations: Item-to-Item Collaborative Filtering. *IEEE Internet Computing*, v. 7, n. 1, p. 76-80, 2003. IEEE: <https://ieeexplore.ieee.org/document/1167344>.

SHAPIRO, C.; VARIAN, H. R. *Information Rules: A Strategic Guide to the Network Economy*. Boston: Harvard Business Review Press, 1999. <https://yunus.hacettepe.edu.tr/~tonta/yayinlar/hal-varian-information-rules-chapter-1.pdf>

CARVALHO, R. S.; SANTOS, P. R. Métricas de Avaliação em Sistemas de Recomendação: Uma Análise Comparativa. In: CONGRESSO BRASILEIRO DE SISTEMAS DE INFORMAÇÃO, 19., 2024, São Paulo. *Anais eletrônicos [...]*. Porto Alegre: Sociedade Brasileira de Computação, 2024. p. 234-245. Disponível em: <https://sol.sbc.org.br/index.php/cbsi/article/view/25634>. Acesso em: 29 out. 2025.

MELO, P. A.; RIBEIRO, T. S. Otimização de Hiperparâmetros em Sistemas de Recomendação usando Grid Search. In: SIMPÓSIO BRASILEIRO DE COMPUTAÇÃO

APLICADA, 12., 2024, Belo Horizonte. Anais eletrônicos [...]. Porto Alegre: Sociedade Brasileira de Computação, 2024. p. 567-578. Disponível em: https://sol.sbc.org.br/index.php/sbca_estendido/article/view/23876. Acesso em: 29 out. 2025.

ROCHA, F. N.; ALVES, D. C. Python para Ciência de Dados: Implementação de Sistemas de Recomendação com Surprise. In: WORKSHOP DE FERRAMENTAS E APLICAÇÕES, 8., 2024, Recife. Anais eletrônicos [...]. Porto Alegre: Sociedade Brasileira de Computação, 2024. p. 123-134. Disponível em: <https://sol.sbc.org.br/index.php/wfa/article/view/24567>. Acesso em: 29 out. 2025.

SOUZA, T. R.; FERREIRA, L. M. Normalização de Dados em Matrizes Usuário-Item: Impacto na Performance de Sistemas de Recomendação. In: ENCONTRO NACIONAL DE INTELIGÊNCIA ARTIFICIAL, 21., 2024, Florianópolis. Anais eletrônicos [...]. Porto Alegre: Sociedade Brasileira de Computação, 2024. p. 345-356. Disponível em: <https://sol.sbc.org.br/index.php/enia/article/view/26789>. Acesso em: 29 out. 2025.

COGALAN, A. Guia Prático para Construção de Sistemas de Recomendação Escaláveis em Python. Medium, 2023. Disponível em: <https://medium.com/@anilcogalan/practical-guide-to-building-scalable-recommender-systems-in-python-b175547e6fce>. Acesso em: 15 mar. 2025.

Apêndices

A) Link para o Github onde estão os gráficos e códigos:

https://github.com/maikisoares00/Projeto_Aplicado_III_RetailWise

B) Link para o Dataset:

<https://archive.ics.uci.edu/dataset/352/online+retail>